

Module 4

UART — UVM+SV Verification

Learning objectives

- UART UVM agent: Transaction (UartTxTransaction), sequence (UartTxSequence), driver (UartTxDriver),...
- Interface: uart_tx_if (clk, rst_n, start, data, tx, baud_tick) connects UVM to DUT.
- Loopback check: TX path (driver → DUT → monitor → scoreboard) and RX path (DUT rx_valid/rx_data → s...

Prerequisites

- Module 1 (spec → RTL), Module 2 (UVM+SV, uvm_smoke), Module 3 (UART protocol + RTL + basic TB).
- Verilator, Make, C++ compiler, UVM (UVM_HOME or vendored UVM).

Learning path

Learning Path

Diagram: Learning Path

(render with mmdc when available)

flowchart LR

T1["UART protocol recap (8N1)"]

T2["Toolchain"]

T1 --> T2

T2 --> N["SPI"]

Extend UART verification to ****UVM+SV**** — UART agent (transaction, sequence, driver, monitor, scoreboard); run on Verilator.

Overview

- Module 4 builds on Module 3 (UART protocol + RTL + basic testbench):

How to learn this module

Suggested learning path

- Read the detailed UART learning guide:
UART_LEARNING_GUIDE.md — what UART is, how it works
(frame,...
- Read the protocols + UVM overview:
LEARNING_GUIDE_PROTOCOLS_AND_UVM.md — Part B § 5.
How UVM Maps t...

Design architecture

1. UART DUT (reused from Module 3)

- dut/: `uart\_tx`, `uart\_rx`,
 `baud\_gen` — identical RTL to
 `uart\_baseline`.
- Loopback in top: `rx = tx` for
 closed-loop checking.
- Why reuse: Upgrade verification
 (UVM) without touching proven
 RTL.

Rtl Architecture

```
Diagram: Rtl Architecture
(render with mmdc when available)
flowchart TB
  subgraph dut["UART DUT (same as Module 3)"]
    BG["baud_gen"]
    TX["uart_tx"]
    RX["uart_rx"]
    BG --> TX
  end
```

2. UVM testbench hierarchy

- uart_tx_if: virtual interface for driver/monitor.
- UartTxTransaction / Sequence / Driver / Monitor / Scoreboard / Agent / Env / Test — same pattern as...
- Monitor reconstructs bytes from the serial `tx` line using UART 8N1 rules.

Verification Architecture

```
Diagram: Verification Architecture
(render with mmdc when available)
flowchart TB
    SEQ["UartTxSequence"] --> DRV["UartTxDriver"]
    DRV --> DUT["uart_tx / uart_rx"]
    DUT --> MON["UartTxMonitor\nreconstruct from tx"]
    MON --> SB["UartTxScoreboard"]
    DUT -->|loopback rx_valid| SB
```

3. Dual check paths

- TX path: sequence → driver → DUT
→ monitor → scoreboard
(`observed\_tx`).
- RX path: loopback into
`uart\_rx`; `check\_rx\_byte()`
when `data\_valid` asserts.

Testing Methods

Diagram: Testing Methods
(render with mmdc when available)
flowchart TB
SEQ["UartTxSequence"] --> DRV["Driver"]
DRV --> DUT["UART DUT"]
DUT --> MON["Monitor on tx"]
MON --> SB["Scoreboard TX"]
DUT --> RX["RX hook"]

RTL block diagram (reference)

Rtl Architecture

Diagram: Rtl Architecture

(render with mmdc when available)

flowchart TB

subgraph dut["UART DUT (same as Module 3)"]

BG["baud_gen"]

TX["uart_tx"]

RX["uart_rx"]

BG --> TX

Module 4: DUT hierarchy and signal flow.

Verification / testbench diagram (reference)

Verification Architecture

Diagram: Verification Architecture

(render with mmdc when available)

flowchart TB

SEQ["UartTxSequence"] --> DRV["UartTxDriver"]

DRV --> DUT["uart_tx / uart_rx"]

DUT --> MON["UartTxMonitor\nreconstruct from tx"]

MON --> SB["UartTxScoreboard"]

DUT -->|loopback rx_valid| SB

Module 4: stimulus, observation, and checking.

UartTxDriver — bit timing

```
txn.data = 8'hFF;  
    start_item(txn);  
    finish_item(txn);  
endtask  
endclass
```

```
//
```

```
-----
```

```
-----
```

```
// Driver
```

```
//
```

```
-----
```

```
-----
```

```
class UartTxDriver extends uvm_driver #(UartTxTransaction);  
    virtual uart_tx_if vif;
```

```
`uvm_component_utils(UartTxDriver)
```

UartTxTransaction & agent

```
/**
 * Module 4: UART TX/RX UVM test (Verilator)
 *
 * UART agent: transaction, sequence, driver, monitor, scoreboard.
 * Loopback: TX output -> RX input; scoreboard checks TX (monitor) and
RX (hook).
 */

`include "uvm_macros.svh"
import uvm_pkg::*;

`timescale 1ns/1ps

//
-----

-----
// Interface
//
```

Execution & simulation flow

How the example runs (toolchain)

- Makefile: Verilator + UVM_HOME compile RTL, interface, and test_*.sv
- UVM phases: build → connect → run_test → report (same pattern as Module 2)
- Sequence → driver → DUT → monitor → scoreboard (read SCOREBOARD at end)
- Typical command: make SIM=verilator TEST=test_* (see module4 demo slide)
- Loopback / hooks connect monitor observations to scoreboard checks

Example directory layout

- dut/: uart_tx.v, uart_rx.v, baud_gen.v (same as Module 3)
- test_uart_uvm.sv: Interface, UartTxTransaction, UartTxSequence, UartTxDriver, UartTxMonitor, UartTx...

Makefile — UVM UART sim

```
# Makefile for Module 4 UART UVM test
#
# Usage:
#   make SIM=verilator TEST=test_uart_uvm
#

SIM ?= verilator
TEST ?= test_uart_uvm

# Prefer externally-set UVM_HOME, but fall back to vendored UVM.
# From module4/examples/uart_uvm, repo root is ../../..
UVM_HOME ?= $(shell echo $$UVM_HOME)
UVM_HOME_FALLBACK := ../../../../tools/uvm-2017/1800.2-2017-1.0
UVM_HOME_FALLBACK_ALT :=
../../../../tools/learn_uvm2017_sv_verilator/tools/uvm-2017/1800.2-2017-1.0
```

Key files to study

Open these in the repo

- ``module4/examples/uart_uvm/dut/`` — same UART RTL as Module 3
- ``module4/examples/uart_uvm/test_uart_uvm.sv`` — agent, env, loopback hooks
- ``docs/LEARNING_GUIDE_PROTOCOLS_AND_UVM.md`` — § 6 UART UVM mapping

Verification & testing methods

1. Directed UART regression

- Sequence sends 0x00, 0x01, 0x55, 0xAA, 0xFF — repeatable regression before randomization.

Testing Methods

```
Diagram: Testing Methods
(render with mmdc when available)
flowchart TB
    SEQ["UartTxSequence"] --> DRV["Driver"]
    DRV --> DUT["UART DUT"]
    DUT --> MON["Monitor on tx"]
    MON --> SB["Scoreboard TX"]
    DUT --> RX["RX hook"]
```

2. Scoreboard strategy

- Expected queue loaded in test;
compare to monitor-reconstructed
TX and to RX hook data.
- Fail fast on mismatch; read UVM
SCOREBOARD summary at end of
run.

Testing Methods

Diagram: Testing Methods

(render with mmdc when available)

flowchart TB

SEQ["UartTxSequence"] --> DRV["Driver"]

DRV --> DUT["UART DUT"]

DUT --> MON["Monitor on tx"]

MON --> SB["Scoreboard TX"]

DUT --> RX["RX hook"]

3. What to add next

- Random baud, framing-error injection, coverage on `busy` / `data\_valid`.
- See
[LEARNING_GUIDE_PROTOCOLS_AND_UVM.md § 6]
(LEARNING_GUIDE_PROTOCOLS_AND_UVM.md#6-uart-uvm-mappin...

Testing Methods

```
Diagram: Testing Methods
(render with mmdc when available)
flowchart TB
    SEQ["UartTxSequence"] --> DRV["Driver"]
    DRV --> DUT["UART DUT"]
    DUT --> MON["Monitor on tx"]
    MON --> SB["Scoreboard TX"]
    DUT --> RX["RX hook"]
```

Scoreboard — TX/RX check

```
imp = new("imp", this);
endfunction

function void add_expected(UartTxTransaction txn);
    expected_queue.push_back(txn);
endfunction

function void write(UartTxTransaction txn);
    UartTxTransaction exp;
    if (expected_queue.size() == 0) begin
        `uvm_warning("SCOREBOARD", "No expected transaction
available")
        return;
    end

    exp = expected_queue.pop_front();
```

Syllabus topics

1. UART protocol recap (8N1)

- One byte = start (0) + 8 data (LSB first) + stop (1); ``baud_tick`` times each bit.
- Driver/monitor must follow the same rules as the RTL.

2. Toolchain

- Verilator ``-sv``, ``--timing``, UVM includes; ``make`
SIM=verilator TEST=test_uart_uvm`; ``+UVM_TESTNAME=...`

Hands-on examples

Module 4 self-check

```
$ ./scripts/module4.sh --help
```

Terminal: module4_check

```
[[0;36m===== [[0m
[[0;36mModule 4: Demo commands (UART UVM) [[0m
[[0;36m===== [[0m

From repo root:

# 1) Environment sanity:
verilator --version
make --version
echo "$UVM_HOME"

# 2) Run the UART UVM example (inside the example directory):
cd module4/examples/uart_uvm
make SIM=verilator TEST=test_uart_uvm

# 3) Inspect logs and waveforms:
ls obj_dir
# If a VCD is produced:
ls *.vcd

[[1;33m[INFO] [[0m See module4/EXAMPLES.md and module4/examples/uart_uvm/README.md for more.
```

Example 1: UART UVM agent

- Same UART DUT as Module 3 with a full UVM agent (sequence, driver, monitor, scoreboard)
- Monitor reconstructs bytes from the serial `tx` line
- Loopback RX checking via `check\_rx\_byte` in the scoreboard

Demo: UART UVM agent

```
$ cd module4/examples/uart_uvm && make SIM=verilator  
TEST=test_uart_uvm
```

Terminal: ex01_uart_uvm

What to look for

- UVM phases and DRIVER, MONITOR, SCOREBOARD messages
- TX and RX scoreboard checks (TX: monitor observes serial line; RX: hook from DUT rx_valid/rx_data)

Key concepts

1. Reuse DUT, upgrade TB

- RTL unchanged from baseline; UVM replaces directed ``initial`` stimulus.

2. Dual path checking

- TX monitor reconstructs serial bytes; RX hook checks loopback ``data_valid``.

Common pitfalls

Monitor baud mismatch

- Mistake: Sampling ``tx`` at system clock instead of ``baud_tick`` times.
- Correct: One sample per ``baud_tick`` per UART rules (as in `UartTxMonitor`).
- Why: Reconstructed byte will not match driven data.

Forgetting loopback RX check

- Mistake: Scoreboard only compares TX monitor output.
- Correct: Also call ``check_rx_byte`` when ``data_valid`` asserts.
- Why: TX path can pass while RX is broken.

Practice & assessment

What you should know

- Describe the UART UVM agent (transaction, sequence, driver, monitor, scoreboard).
- Explain loopback (TX→RX) and how TX and RX are both checked in the scoreboard.
- Run the `uart_uvm` example and interpret UVM output.
- Ready for Module 5: SPI protocol + RTL + basic testbench.

Exercises

- Run uart_uvm
- Trace one transaction
- Change the sequence
- Optional: Loopback path

Assessment checklist

- Can describe the UART UVM agent: transaction (one byte), sequence, driver, monitor, scoreboard.
- Can explain loopback and how TX and RX are both checked.
- Can run `uart_uvm` and interpret UVM output (phases, scoreboard summary).
- Ready for Module 5: SPI protocol + RTL + basic testbench.

Summary & next steps

- Extend UART verification to ****UVM+SV**** — UART agent (transaction, sequence, driver, monitor, scoreb...
- Complete module4/CHECKLIST.md
- Review module4/EXAMPLES.md and run each lab
- Next: SPI